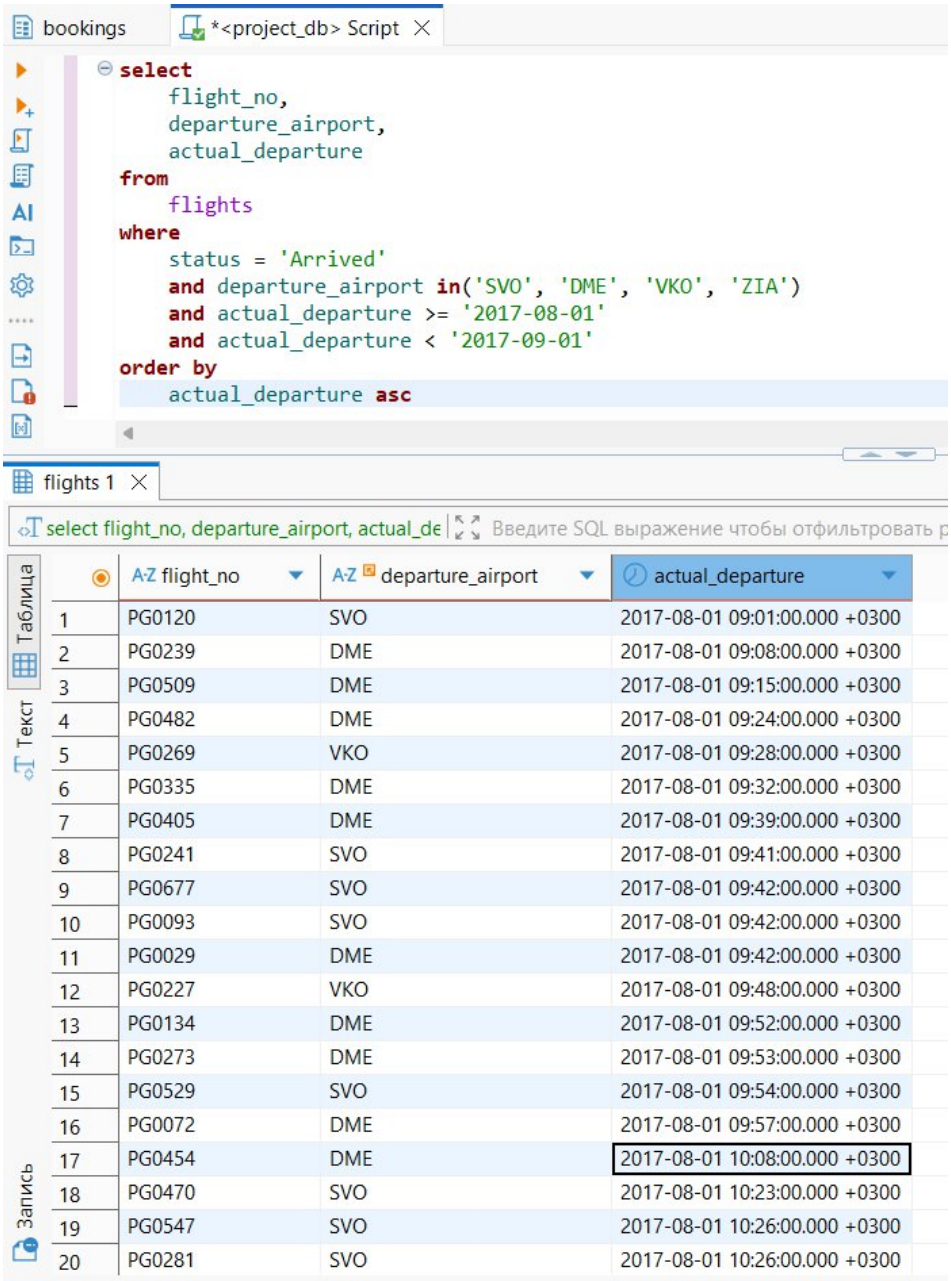
ДЗ 4

- **Задача 1.1.** Выбрать все выполненные рейсы (status = 'Arrived') из московских аэропортов за август 2017 года: номер рейса, аэропорт вылета, фактическое время вылета. Отсортировать по возрастанию времени вылета.



	flight_no	departure_airport	actual_departure
1	PG0120	SVO	2017-08-01 09:01:00.000 +0300
2	PG0239	DME	2017-08-01 09:08:00.000 +0300
3	PG0509	DME	2017-08-01 09:15:00.000 +0300
4	PG0482	DME	2017-08-01 09:24:00.000 +0300
5	PG0269	VKO	2017-08-01 09:28:00.000 +0300
6	PG0335	DME	2017-08-01 09:32:00.000 +0300
7	PG0405	DME	2017-08-01 09:39:00.000 +0300
8	PG0241	SVO	2017-08-01 09:41:00.000 +0300
9	PG0677	SVO	2017-08-01 09:42:00.000 +0300
10	PG0093	SVO	2017-08-01 09:42:00.000 +0300
11	PG0029	DME	2017-08-01 09:42:00.000 +0300
12	PG0227	VKO	2017-08-01 09:48:00.000 +0300
13	PG0134	DME	2017-08-01 09:52:00.000 +0300
14	PG0273	DME	2017-08-01 09:53:00.000 +0300
15	PG0529	SVO	2017-08-01 09:54:00.000 +0300
16	PG0072	DME	2017-08-01 09:57:00.000 +0300
17	PG0454	DME	2017-08-01 10:08:00.000 +0300
18	PG0470	SVO	2017-08-01 10:23:00.000 +0300
19	PG0547	SVO	2017-08-01 10:26:00.000 +0300
20	PG0281	SVO	2017-08-01 10:26:00.000 +0300

- **Задача 1.2.** Найти рейсы, прибывшие с опозданием более чем на 2 часа: номер рейса, плановое и фактическое время прибытия, опоздание в минутах. Самые большие опоздания — сверху.

```sql
select
    flight_no,
    scheduled_arrival,
    actual_arrival,
    extract (epoch from (actual_arrival - scheduled_arrival))/ 60 as delay_minutes
from
    flights
where
    actual_arrival is not null
    and actual_arrival - scheduled_arrival > interval '2 hours'
order by delay_minutes desc
```

flights 1

select flight_no, scheduled_arrival, actual_arr | Введите SQL выражение чтобы отфильтровать результаты

	flight_no	scheduled_arrival	actual_arrival	delay_minutes
1	PG0531	2017-05-17 11:45:00.000 +0300	2017-05-17 16:26:00.000 +0300	281
2	PG0132	2017-05-22 13:55:00.000 +0300	2017-05-22 18:36:00.000 +0300	281
3	PG0589	2017-07-29 15:55:00.000 +0300	2017-07-29 20:32:00.000 +0300	277
4	PG0364	2017-07-19 13:20:00.000 +0300	2017-07-19 17:49:00.000 +0300	269
5	PG0151	2017-07-03 14:30:00.000 +0300	2017-07-03 18:57:00.000 +0300	267
6	PG0337	2017-06-01 14:10:00.000 +0300	2017-06-01 18:36:00.000 +0300	266
7	PG0354	2017-07-03 10:10:00.000 +0300	2017-07-03 14:36:00.000 +0300	266
8	PG0544	2017-07-06 12:05:00.000 +0300	2017-07-06 16:31:00.000 +0300	266
9	PG0164	2017-07-29 18:35:00.000 +0300	2017-07-29 23:00:00.000 +0300	265
10	PG0635	2017-06-24 13:15:00.000 +0300	2017-06-24 17:39:00.000 +0300	264
11	PG0483	2017-06-24 11:20:00.000 +0300	2017-06-24 15:44:00.000 +0300	264
12	PG0650	2017-05-19 14:00:00.000 +0300	2017-05-19 18:22:00.000 +0300	262
13	PG0444	2017-05-25 14:00:00.000 +0300	2017-05-25 18:22:00.000 +0300	262
14	PG0568	2017-08-13 17:30:00.000 +0300	2017-08-13 21:49:00.000 +0300	259
15	PG0096	2017-08-03 18:50:00.000 +0300	2017-08-03 23:08:00.000 +0300	258
16	PG0454	2017-08-02 10:40:00.000 +0300	2017-08-02 14:58:00.000 +0300	258
17	PG0214	2017-06-20 15:35:00.000 +0300	2017-06-20 19:52:00.000 +0300	257
18	PG0050	2017-05-26 15:20:00.000 +0300	2017-05-26 19:36:00.000 +0300	256
19	PG0564	2017-08-10 12:45:00.000 +0300	2017-08-10 17:01:00.000 +0300	256
20	PG0538	2017-05-22 12:45:00.000 +0300	2017-05-22 17:01:00.000 +0300	256
21	PG0166	2017-08-12 15:10:00.000 +0300	2017-08-12 19:26:00.000 +0300	256
22	PG0306	2017-06-09 15:30:00.000 +0300	2017-06-09 19:46:00.000 +0300	256

Обновить | Save | Cancel | Экспорт данных ... | 200

200 строк получено - 0,055s (0,002s получ.), 2026-10-11 в 11:30:56

EXTRACT(EPOCH FROM ...) берёт интервал и возвращает общее количество секунд

- **Задача 1.3.** Рейсы с номерами, начинающимися на PG04 (номера диапазона PG0400–PG0499): сколько выполнено рейсов каждым номером.

```sql
select
    flight_no,
    count(*) as flight_count
from
    flights
where
    status = 'Arrived'
    and flight_no between 'PG0400' and 'PG0499'
group by flight_no
order by flight_no
```

lights 1

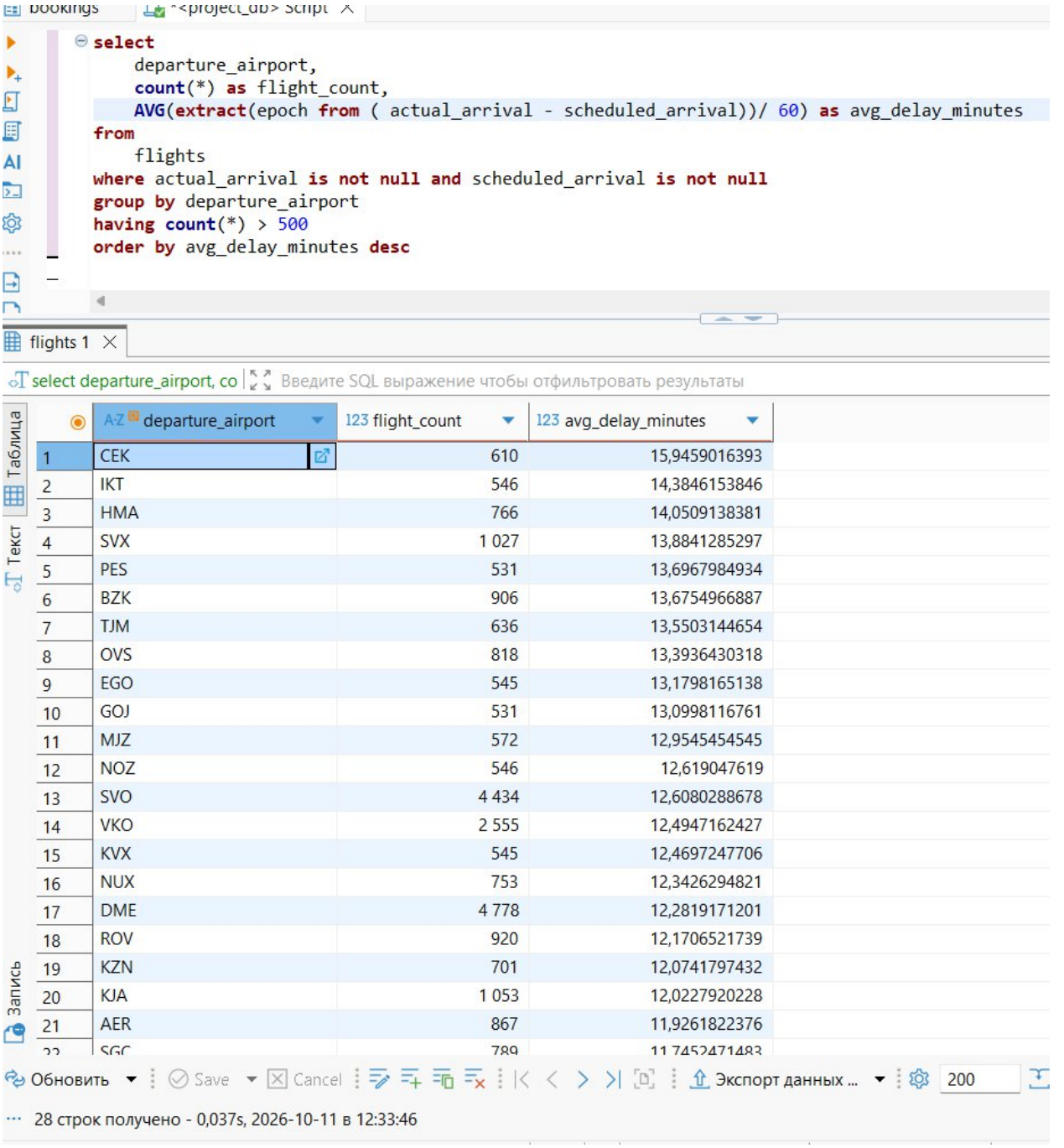
elect flight_no, count(*) as flight_count fron | Введите SQL выражение

	A-Z flight_no	123 flight_count
1	PG0400	91
2	PG0401	91
3	PG0402	91
4	PG0403	91
5	PG0404	90
6	PG0405	91
7	PG0406	90
8	PG0407	91
9	PG0408	91
10	PG0409	91
11	PG0410	91
12	PG0411	91
13	PG0412	91
14	PG0413	90
15	PG0414	91
16	PG0415	91
17	PG0416	90
18	PG0417	90
19	PG0418	91
20	PG0419	91
21	PG0420	13
22	PG0421	13

)бновить Save Cancel

100 строк получено - 0,026s, 2026-10-11 в 11:54:45

- **Задача 2.1.** Для каждого аэропорта вылета: число рейсов и среднее опоздание прибытия в минутах. Оставить только аэропорты с более чем 500 рейсами, отсортировать по убыванию среднего опоздания.



```sql
select
    departure_airport,
    count(*) as flight_count,
    AVG(extract(epoch from ( actual_arrival - scheduled_arrival))/ 60) as avg_delay_minutes
from
    flights
where actual_arrival is not null and scheduled_arrival is not null
group by departure_airport
having count(*) > 500
order by avg_delay_minutes desc
```

flights 1

	departure_airport	flight_count	avg_delay_minutes
1	CEK	610	15,9459016393
2	IKT	546	14,3846153846
3	HMA	766	14,0509138381
4	SVX	1 027	13,8841285297
5	PES	531	13,6967984934
6	BZK	906	13,6754966887
7	TJM	636	13,5503144654
8	OVS	818	13,3936430318
9	EGO	545	13,1798165138
10	GOJ	531	13,0998116761
11	MJZ	572	12,9545454545
12	NOZ	546	12,619047619
13	SVO	4 434	12,6080288678
14	VKO	2 555	12,4947162427
15	KVX	545	12,4697247706
16	NUX	753	12,3426294821
17	DME	4 778	12,2819171201
18	ROV	920	12,1706521739
19	KZN	701	12,0741797432
20	KJA	1 053	12,0227920228
21	AER	867	11,9261822376
22	SGC	789	11,7452471483

Обновить | Save | Cancel | Экспорт данных ... | 200

28 строк получено - 0,037s, 2026-10-11 в 12:33:46

- **Задача 3.1.** Топ-10 самых дорогих проданных билетов: имя пассажира, сумма, номер рейса, город вылета и город прилёта.

```sql
select
    t.passenger_name,
    tf.amount,
    f.flight_no,
    dep.city AS departure_city,
    arr.city AS arrival_city
from
    tickets as t
join ticket_flights tf on t.ticket_no = tf.ticket_no
join flights f on tf.flight_id = f.flight_id
join airports_data dep on f.departure_airport = dep.airport_code
join airports_data arr on f.arrival_airport = arr.airport_code
order by tf.amount desc
limit 10
```

tickets(+) 1

select t.passenger_name, tf.amount, f.flight_r | Введите SQL выражение чтобы отфильтровать результаты

	passenger_name	amount	flight_no	departure_city	arrival_city
1	ANTON PETROV	203 300	PG0144	{"en": "Moscow", "ru": "Москва	{"en": "Petropavlovsk", "ru": "П
2	YAKOV CHERNOV	203 300	PG0144	{"en": "Moscow", "ru": "Москва	{"en": "Petropavlovsk", "ru": "П
3	ANASTASIYA MOROZOVA	203 300	PG0144	{"en": "Moscow", "ru": "Москва	{"en": "Petropavlovsk", "ru": "П
4	RAISA KAZAKOVA	203 300	PG0144	{"en": "Moscow", "ru": "Москва	{"en": "Petropavlovsk", "ru": "П
5	LYUBOV FROLOVA	203 300	PG0144	{"en": "Moscow", "ru": "Москва	{"en": "Petropavlovsk", "ru": "П
6	OLGA KOZLOVA	203 300	PG0144	{"en": "Moscow", "ru": "Москва	{"en": "Petropavlovsk", "ru": "П
7	LIDIYA ZHUKOVA	203 300	PG0145	{"en": "Petropavlovsk", "ru": "П	{"en": "Moscow", "ru": "Москва
8	OLEG MIRONOV	203 300	PG0145	{"en": "Petropavlovsk", "ru": "П	{"en": "Moscow", "ru": "Москва
9	LYUBOV ALEKSANDROVA	203 300	PG0145	{"en": "Petropavlovsk", "ru": "П	{"en": "Moscow", "ru": "Москва
10	SERGEY KISELEV	203 300	PG0145	{"en": "Petropavlovsk", "ru": "П	{"en": "Moscow", "ru": "Москва

- **Задача 3.2.** Вывести все аэропорты с количеством вылетающих из них рейсов — включая аэропорты, из которых вообще ничего не летает.

```sql
select
    a.airport_name,
    a.airport_code,
    a.city,
    count(f.flight_id) as flight_count
from
    airports_data as a
left join flights f on f.departure_airport = a.airport_code
group by a.airport_name, a.airport_code,a.city
order by flight_count desc,a.airport_code
```

airports_data 1

select a.airport_name, a.airp Введите SQL выражение чтобы отфильтровать результаты

	airport_name	airport_code	city	flight_count
1	{"en": "Domodedovo Internati	DME	{"en": "Moscow", "ru": "Москва	6 376
2	{"en": "Sheremetyevo Internati	SVO	{"en": "Moscow", "ru": "Москва	5 912
3	{"en": "Pulkovo Airport", "ru": "	LED	{"en": "St. Petersburg", "ru": "С	3 769
4	{"en": "Vnukovo International A	VKO	{"en": "Moscow", "ru": "Москва	3 405
5	{"en": "Tolmachevo Airport", "r	OVB	{"en": "Novosibirsk", "ru": "Нов	2 091
6	{"en": "Yemelyanovo Airport",	KJA	{"en": "Krasnoyarsk", "ru": "Кра	1 401
7	{"en": "Koltsovo Airport", "ru":	SVX	{"en": "Yekaterinburg", "ru": "Е	1 367
8	{"en": "Bolshoye Savino Airpor	PEE	{"en": "Perm", "ru": "Пермь"}	1 228
9	{"en": "Rostov-on-Don Airport	ROV	{"en": "Rostov", "ru": "Ростов-н	1 227
10	{"en": "Bryansk Airport", "ru": "	BZK	{"en": "Bryansk", "ru": "Брянск"	1 210
11	{"en": "Sochi International Airp	AER	{"en": "Sochi", "ru": "Сочи"}	1 159
12	{"en": "Sovetskiy Airport", "ru":	OVS	{"en": "Sovetskiy", "ru": "Совет	1 089
13	{"en": "Surgut Airport", "ru": "С	SGC	{"en": "Surgut", "ru": "Сургут"}	1 055
14	{"en": "Khanty Mansiysk Airpor	HMA	{"en": "Khanty-Mansiysk", "ru":	1 021
15	{"en": "Novy Urengoy Airport",	NUX	{"en": "Novy Urengoy", "ru": "Н	1 003
16	{"en": "Kazan International Airp	KZN	{"en": "Kazan", "ru": "Казань"}	934
17	{"en": "Roshchino Internationa	TJM	{"en": "Tyumen", "ru": "Тюмен	847
18	{"en": "Chelyabinsk Balandino	CEK	{"en": "Chelyabinsk", "ru": "Чел	812
19	{"en": "Khabarovsk-Novy Airpo	KHV	{"en": "Khabarovsk", "ru": "Хаб	795

Обновить Save Cancel Экспорт данных ... 200

104 строк получено - 0,043s, 2026-10-11 в 13:00:41

- **Задача 3.3.** Топ-5 пассажиров по суммарным тратам на билеты: id пассажира, имя, число перелётов, суммарная стоимость.

bookings *<project_db> Script

```
select
    t.passenger_id,
    t.passenger_name,
    SUM(tf.amount) as sum_tickets,
    count(*) as flights_count
from
    tickets t
join ticket_flights tf on t.ticket_no = tf.ticket_no
group by t.passenger_id,t.passenger_name
order by sum_tickets desc
limit 5
```

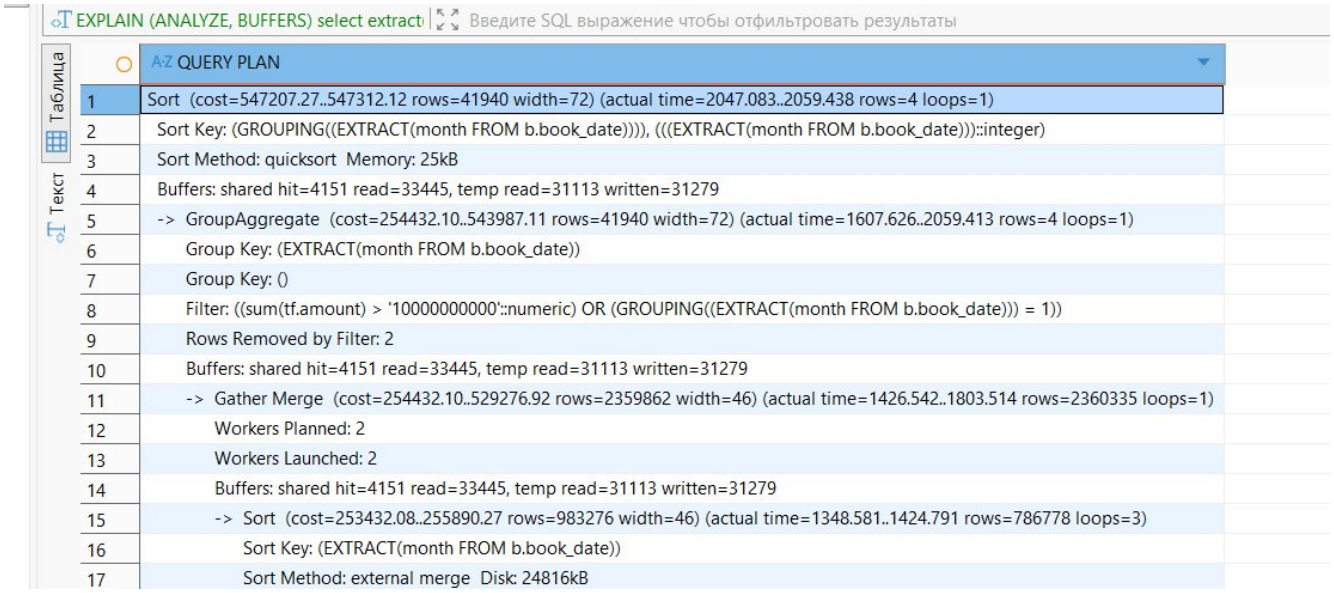
tickets 1

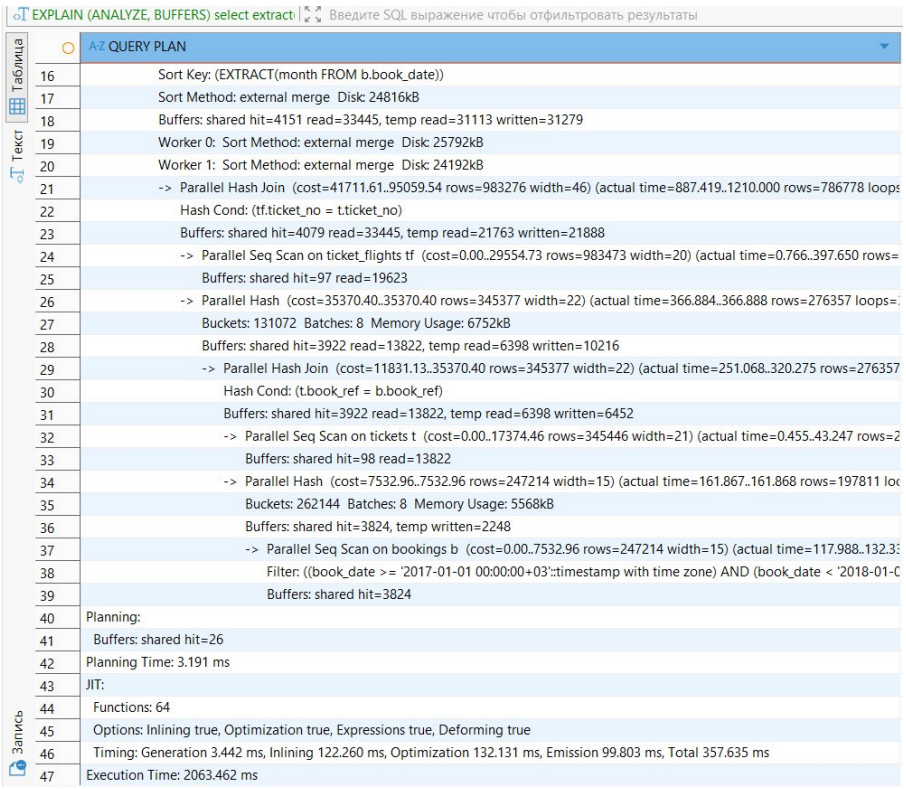
select t.passenger_id, t.pass | Введите SQL выражение чтобы отфильтровать результаты

	passenger_id	passenger_name	sum_tickets	flights_count
1	2034 201228	ANTONINA KUZNECOVA	529 500	6
2	3272 478491	EKATERINA LAZAREVA	506 700	6
3	6615 949899	ELENA VASILEVA	505 500	6
4	7989 541468	EVGENIY TARASOV	495 300	6
5	4805 339011	DMITRIY KUZMIN	494 700	6

1)

```
EXPLAIN (ANALYZE, BUFFERS)
select
        extract(month from b.book_date)::int as month,
        sum(tf.amount) as vyruchka
from bookings as b
join tickets as t on t.book_ref = b.book_ref
join ticket_flights as tf on tf.ticket_no = t.ticket_no
where b.book_date >= '2017-01-01' and b.book_date < '2018-01-01'
group by rollup (extract(month from b.book_date))
having sum(tf.amount) > 10000000000 or grouping(extract(month from b.book_date))=1
order by grouping(extract(month from b.book_date)), month
```





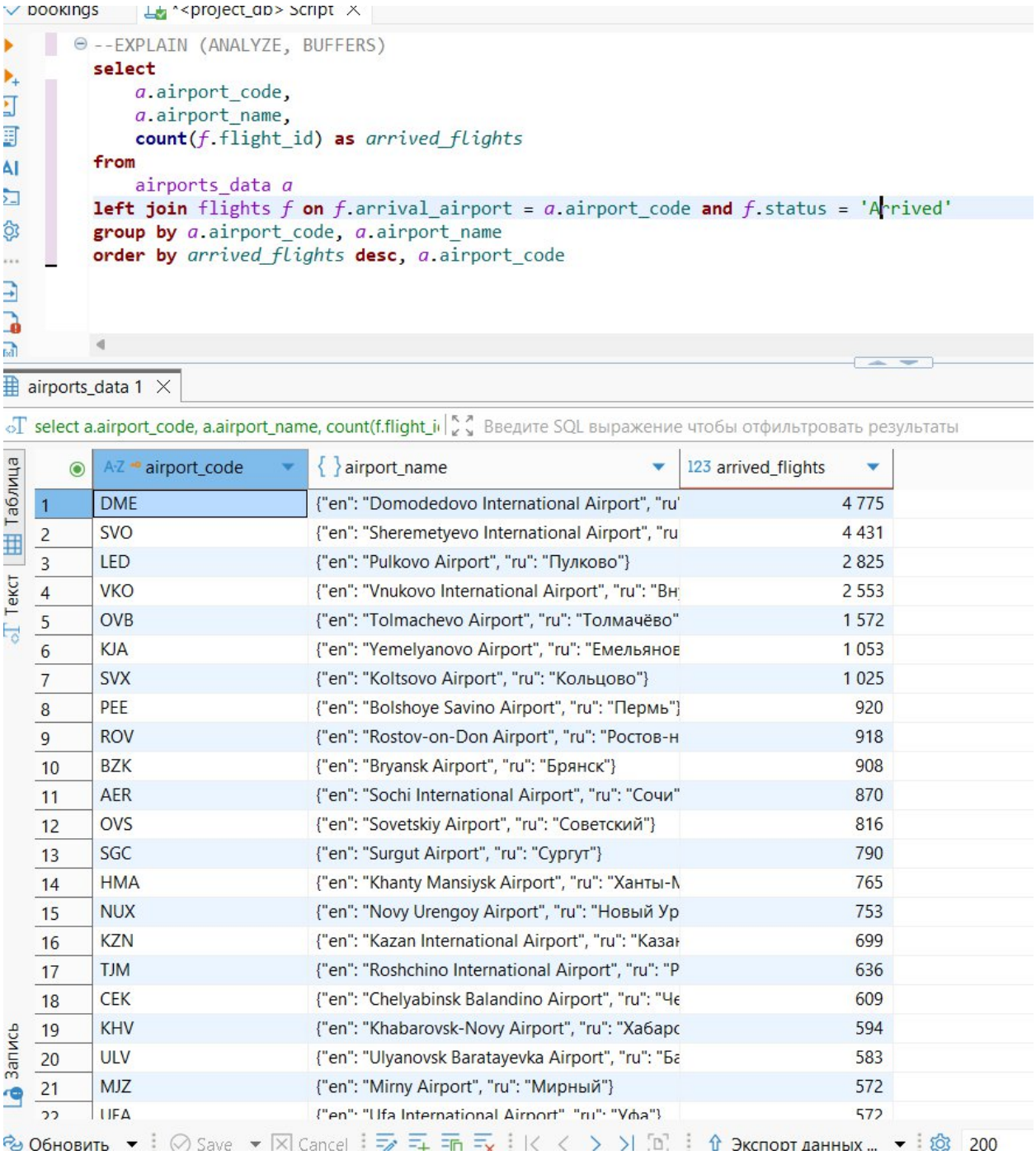
Запрос работал в основном с диском: в буферах есть чтение (shared read ≈ 37 500 страниц) при почти полном отсутствии попаданий в кеш (shared hit = 134), а сортировка и хеш-таблицы ушли во временные файлы (temp read/written ≈ 31 000 страниц). Индексы не использовались, так как все три таблицы читались через Parallel Seq Scan, что можно объяснить низкой селективностью условий WHERE. Кеш задействован минимально, поэтому почти все данные поднимались с диска.

1. *как HAVING работает вместе с ROLLUP и почему итоговая строка (со значением NULL в колонке месяца) проходит фильтр.*

   Rollup сначала группирует месяца, потом добавляет еще одну итоговую группу по всем строкам – в колонке месяца стоит null, а сумма у нас считается по всему году. Having выполняется уже после группировки, поэтому он фильтрует группы, а не исходные строки.
   Итоговая строка проходит фильтр, так как условие SUM(tf.amount) > 10000000000 для неё само по себе может быть истинным или ложным, но добавленное or grouping(month) = 1 гарантирует, что строка с null в месяце всегда проходит having. grouping () возвращает 1 именно для итоговой строки ROLLUP и 0 для обычных месяцев, поэтому итог не теряется даже если его сумма окажется ниже порога.

2

Запуск с ON


bookings
*<project_db> Script
--EXPLAIN (ANALYZE, BUFFERS)
select
a.airport_code,
a.airport_name,
count(f.flight_id) as arrived_flights
from
airports_data a
left join flights f on f.arrival_airport = a.airport_code and f.status = 'Arrived'
group by a.airport_code, a.airport_name
order by arrived_flights desc, a.airport_code
airports_data 1
select a.airport_code, a.airport_name, count(f.flight_i
Введите SQL выражение чтобы отфильтровать результаты
Таблица
Текст
Запись
airport_code
airport_name
arrived_flights
1 DME {"en": "Domodedovo International Airport", "ru" 4 775
2 SVO {"en": "Sheremetyevo International Airport", "ru 4 431
3 LED {"en": "Pulkovo Airport", "ru": "Пулково"} 2 825
4 VKO {"en": "Vnukovo International Airport", "ru": "Вн 2 553
5 OVB {"en": "Tolmachevo Airport", "ru": "Толмачёво" 1 572
6 KJA {"en": "Yemelyanovo Airport", "ru": "Емельянов 1 053
7 SVX {"en": "Koltsovo Airport", "ru": "Кольцово"} 1 025
8 PEE {"en": "Bolshoye Savino Airport", "ru": "Пермь"} 920
9 ROV {"en": "Rostov-on-Don Airport", "ru": "Ростов-н 918
10 BZK {"en": "Bryansk Airport", "ru": "Брянск"} 908
11 AER {"en": "Sochi International Airport", "ru": "Сочи" 870
12 OVS {"en": "Sovetskiy Airport", "ru": "Советский"} 816
13 SGC {"en": "Surgut Airport", "ru": "Сургут"} 790
14 HMA {"en": "Khanty Mansiysk Airport", "ru": "Ханты-М 765
15 NUX {"en": "Novy Urengoy Airport", "ru": "Новый Ур 753
16 KZN {"en": "Kazan International Airport", "ru": "Казан 699
17 TJM {"en": "Roshchino International Airport", "ru": "Р 636
18 CEK {"en": "Chelyabinsk Balandino Airport", "ru": "Че 609
19 KHV {"en": "Khabarovsk-Novy Airport", "ru": "Хабаро 594
20 ULV {"en": "Ulyanovsk Baratayevka Airport", "ru": "Ба 583
21 MJZ {"en": "Mirny Airport", "ru": "Мирный"} 572
22 UFA {"en": "Ufa International Airport", "ru": "Уфа"} 572
Обновить
Save
Cancel
Экспорт данных ...
200

Результат 1

EXPLAIN (analyze, buffers) select a.airport_code, a.ai | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	QUERY PLAN
4	Buffers: shared hit=838
5	-> HashAggregate (cost=2039.52..2040.56 rows=104 width=73) (actual time=51.803..51.816 rows=104 loops=1)
6	Group Key: a.airport_code
7	Batches: 1 Memory Usage: 48kB
8	Buffers: shared hit=835
9	-> Hash Right Join (cost=5.34..1792.89 rows=49325 width=69) (actual time=0.070..42.526 rows=49235 loops=1)
10	Hash Cond: (f.arrival_airport = a.airport_code)
11	Buffers: shared hit=835
12	-> Seq Scan on flights f (cost=0.00..1652.80 rows=49325 width=8) (actual time=0.009..30.672 rows=49235 loops=1)
13	Filter: ((status)::text = 'Arrived'::text)
14	Rows Removed by Filter: 16429
15	Buffers: shared hit=832
16	-> Hash (cost=4.04..4.04 rows=104 width=65) (actual time=0.052..0.053 rows=104 loops=1)
17	Buckets: 1024 Batches: 1 Memory Usage: 18kB
18	Buffers: shared hit=3
19	-> Seq Scan on airports_data a (cost=0.00..4.04 rows=104 width=65) (actual time=0.009..0.026 rows=104 loops=1
20	Buffers: shared hit=3
21	Planning:
22	Buffers: shared hit=6
23	Planning Time: 0.281 ms

С where

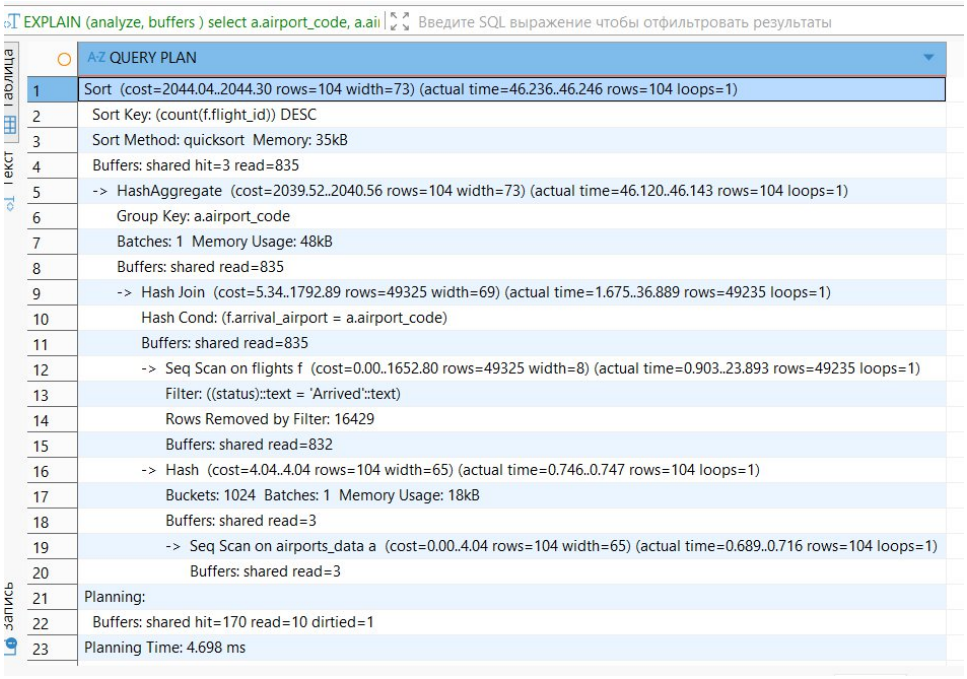
```
--EXPLAIN (analyze, buffers )
select
    a.airport_code,
    a.airport_name,
    count(f.flight_id) as arrived_flights
from airports_data a
left join flights f
    on f.arrival_airport = a.airport_code
where f.status = 'Arrived'
group by a.airport_code, a.airport_name
order by arrived_flights desc
```

ts_data 1

: a.airport_code, a.airport_name, count(f.flight_i | Введите SQL выражение чтобы отфильтровать результаты

airport_code	airport_name	arrived_flights
DME	{"en": "Domodedovo International Airport", "ru": "Домодедово"}	4 775
SVO	{"en": "Sheremetyevo International Airport", "ru": "Шереметьево"}	4 431
LED	{"en": "Pulkovo Airport", "ru": "Пулково"}	2 825
VKO	{"en": "Vnukovo International Airport", "ru": "Внуково"}	2 553
OVB	{"en": "Tolmachevo Airport", "ru": "Толмачёво"}	1 572
KJA	{"en": "Yemelyanovo Airport", "ru": "Емельяново"}	1 053
SVX	{"en": "Koltsovo Airport", "ru": "Кольцово"}	1 025
PEE	{"en": "Bolshoye Savino Airport", "ru": "Пермь"}	920
ROV	{"en": "Rostov-on-Don Airport", "ru": "Ростов-на-Дону"}	918
BZK	{"en": "Bryansk Airport", "ru": "Брянск"}	908
AER	{"en": "Sochi International Airport", "ru": "Сочи"}	870
OVS	{"en": "Sovetskiy Airport", "ru": "Советский"}	816
SGC	{"en": "Surgut Airport", "ru": "Сургут"}	790
HMA	{"en": "Khanty Mansiysk Airport", "ru": "Ханты-Мансийск"}	765
NUX	{"en": "Novy Urengoy Airport", "ru": "Новый Уренгой"}	753
KZN	{"en": "Kazan International Airport", "ru": "Казань"}	699
TJM	{"en": "Roshchino International Airport", "ru": "Рощино"}	636
CEK	{"en": "Chelyabinsk Balandino Airport", "ru": "Челябинск"}	609
KHV	{"en": "Khabarovsk-Novy Airport", "ru": "Хабаровск-Новый"}	594
ULV	{"en": "Ulyanovsk Baratayevka Airport", "ru": "Баратаевка"}	583
MJZ	{"en": "Mirny Airport", "ru": "Мирный"}	572
UFA	{"en": "Ufa International Airport", "ru": "Уфа"}	572
NOZ	{"en": "Spichenkovo Airport", "ru": "Спиченково"}	546

зить | Save | Cancel | Экспорт данных ... | 200 | 104 | 104 строк получен



EXPLAIN (analyze, buffers) select a.airport_code, a.aii | Введите SQL выражение чтобы отфильтровать результаты

	QUERY PLAN
1	Sort (cost=2044.04..2044.30 rows=104 width=73) (actual time=46.236..46.246 rows=104 loops=1)
2	Sort Key: (count(f.flight_id)) DESC
3	Sort Method: quicksort Memory: 35kB
4	Buffers: shared hit=3 read=835
5	-> HashAggregate (cost=2039.52..2040.56 rows=104 width=73) (actual time=46.120..46.143 rows=104 loops=1)
6	Group Key: a.airport_code
7	Batches: 1 Memory Usage: 48kB
8	Buffers: shared read=835
9	-> Hash Join (cost=5.34..1792.89 rows=49325 width=69) (actual time=1.675..36.889 rows=49235 loops=1)
10	Hash Cond: (f.arrival_airport = a.airport_code)
11	Buffers: shared read=835
12	-> Seq Scan on flights f (cost=0.00..1652.80 rows=49325 width=8) (actual time=0.903..23.893 rows=49235 loops=1)
13	Filter: ((status)::text = 'Arrived'::text)
14	Rows Removed by Filter: 16429
15	Buffers: shared read=832
16	-> Hash (cost=4.04..4.04 rows=104 width=65) (actual time=0.746..0.747 rows=104 loops=1)
17	Buckets: 1024 Batches: 1 Memory Usage: 18kB
18	Buffers: shared read=3
19	-> Seq Scan on airports_data a (cost=0.00..4.04 rows=104 width=65) (actual time=0.689..0.716 rows=104 loops=1)
20	Buffers: shared read=3
21	Planning:
22	Buffers: shared hit=170 read=10 dirtied=1
23	Planning Time: 4.698 ms

с ON аэропорт без рейсов остаётся с нулём, а с WHERE LEFT по сути становится INNER, и такой аэропорт выкидывается. Аэропортов без рейсов нет, значит нулей нет и выкидывать нечего, поэтому таблицы совпадают. Разницу на полных данных видно только в EXPLAIN: с ON там Hash Right Join, с WHERE обычный Hash Join. То есть планировщик сам превратил запрос в inner.

COUNT(*) считает строки результата, а не непустые значения. Для аэропорта без рейсов LEFT JOIN всё равно даёт одну строку (с NULL в полях flights).
Значит, COUNT(*) вернёт 1 вместо правильного 0 — то есть «прибыл один рейс» там, где не прибыло ни одного. COUNT(f.flight_id) игнорирует NULL и даёт корректный 0.

Hash Right Join -- это способ физического соединения. PostgreSQL строит хеш-таблицу по одной стороне, затем сканирует вторую сторону (flights) и для каждой строки ищет совпадение по ключу arrival_airport = airport_code.
Слово «Right» отражает, что PostgreSQL внутренне перевернул LEFT JOIN в RIGHT JOIN -- так удобнее строить хеш. Семантика результата при этом сохраняется. Условие status = 'Arrived' применяется на этапе сканирования flights (в Filter), то есть до соединения -- поэтому аэропорты без подходящих рейсов не отсеиваются и сохраняются с NULL

Краткое резюме:

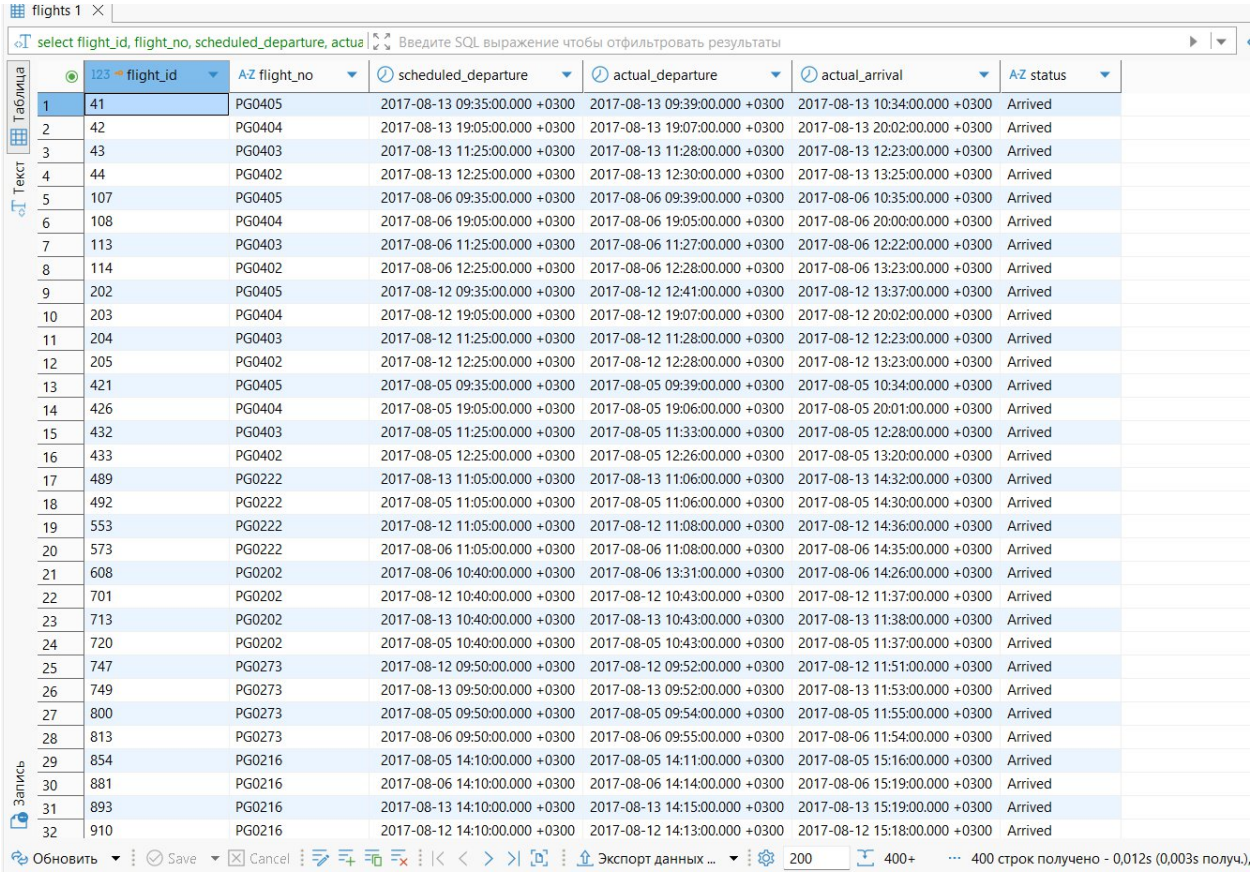
В Buffers указано shared hit — запрос работал с кешем, чтения с диска (shared read) не было.

Индекс не использовался: обе таблицы читались через Seq
Scan (последовательное сканирование), потому что фильтр по статусу невысокой селективности, а подходящего индекса по flights.arrival_airport с учётом status нет.

3

Неоптимизированный запрос

```
select
    flight_id,
    flight_no,
    scheduled_departure,
    actual_departure,
    actual_arrival,
    status
from flights
where
    status = 'Arrived'
    and extract(DOW FROM scheduled_departure) in (0, 6)      -- 0 = воскресенье, 6 = суббота
    and extract (MONTH FROM scheduled_departure) = 8
    and extract(YEAR  FROM scheduled_departure) = 2017
```



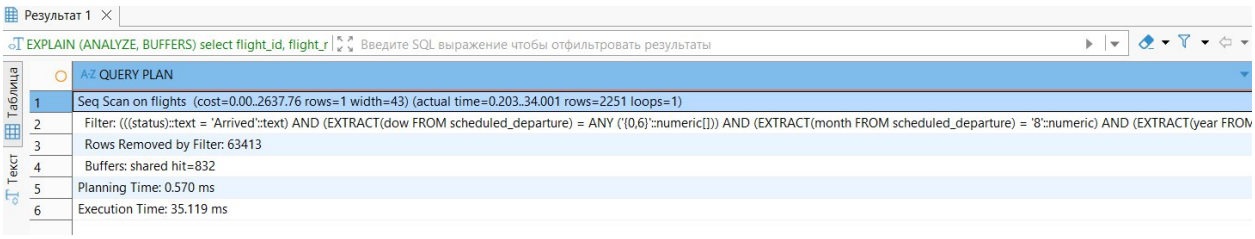
flights 1

select flight_id, flight_no, scheduled_departure, actua | Введите SQL выражение чтобы отфильтровать результаты

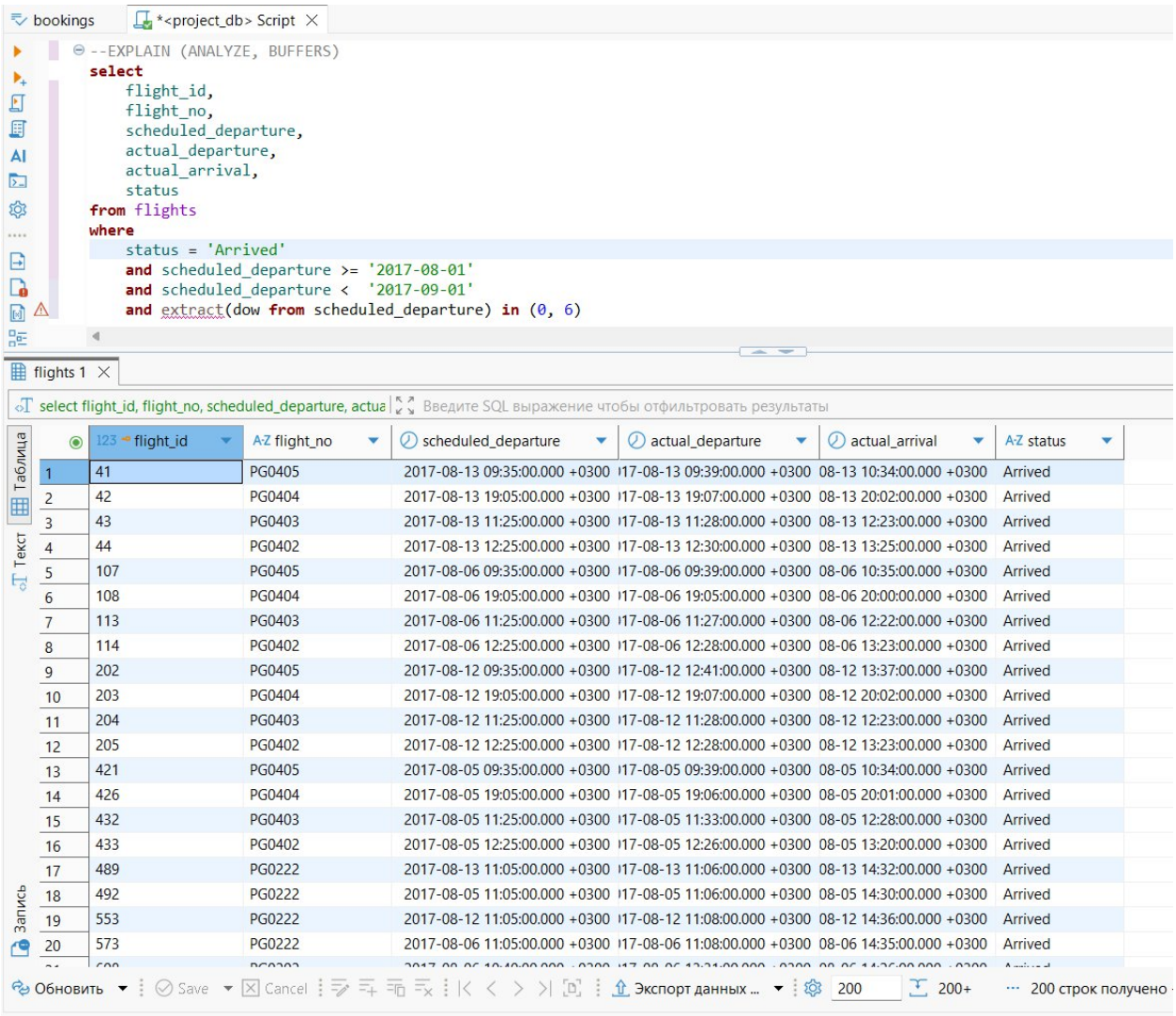
	flight_id	flight_no	scheduled_departure	actual_departure	actual_arrival	status
1	41	PG0405	2017-08-13 09:35:00.000 +0300	2017-08-13 09:39:00.000 +0300	2017-08-13 10:34:00.000 +0300	Arrived
2	42	PG0404	2017-08-13 19:05:00.000 +0300	2017-08-13 19:07:00.000 +0300	2017-08-13 20:02:00.000 +0300	Arrived
3	43	PG0403	2017-08-13 11:25:00.000 +0300	2017-08-13 11:28:00.000 +0300	2017-08-13 12:23:00.000 +0300	Arrived
4	44	PG0402	2017-08-13 12:25:00.000 +0300	2017-08-13 12:30:00.000 +0300	2017-08-13 13:25:00.000 +0300	Arrived
5	107	PG0405	2017-08-06 09:35:00.000 +0300	2017-08-06 09:39:00.000 +0300	2017-08-06 10:35:00.000 +0300	Arrived
6	108	PG0404	2017-08-06 19:05:00.000 +0300	2017-08-06 19:05:00.000 +0300	2017-08-06 20:00:00.000 +0300	Arrived
7	113	PG0403	2017-08-06 11:25:00.000 +0300	2017-08-06 11:27:00.000 +0300	2017-08-06 12:22:00.000 +0300	Arrived
8	114	PG0402	2017-08-06 12:25:00.000 +0300	2017-08-06 12:28:00.000 +0300	2017-08-06 13:23:00.000 +0300	Arrived
9	202	PG0405	2017-08-12 09:35:00.000 +0300	2017-08-12 12:41:00.000 +0300	2017-08-12 13:37:00.000 +0300	Arrived
10	203	PG0404	2017-08-12 19:05:00.000 +0300	2017-08-12 19:07:00.000 +0300	2017-08-12 20:02:00.000 +0300	Arrived
11	204	PG0403	2017-08-12 11:25:00.000 +0300	2017-08-12 11:28:00.000 +0300	2017-08-12 12:23:00.000 +0300	Arrived
12	205	PG0402	2017-08-12 12:25:00.000 +0300	2017-08-12 12:28:00.000 +0300	2017-08-12 13:23:00.000 +0300	Arrived
13	421	PG0405	2017-08-05 09:35:00.000 +0300	2017-08-05 09:39:00.000 +0300	2017-08-05 10:34:00.000 +0300	Arrived
14	426	PG0404	2017-08-05 19:05:00.000 +0300	2017-08-05 19:06:00.000 +0300	2017-08-05 20:01:00.000 +0300	Arrived
15	432	PG0403	2017-08-05 11:25:00.000 +0300	2017-08-05 11:33:00.000 +0300	2017-08-05 12:28:00.000 +0300	Arrived
16	433	PG0402	2017-08-05 12:25:00.000 +0300	2017-08-05 12:26:00.000 +0300	2017-08-05 13:20:00.000 +0300	Arrived
17	489	PG0222	2017-08-13 11:05:00.000 +0300	2017-08-13 11:06:00.000 +0300	2017-08-13 14:32:00.000 +0300	Arrived
18	492	PG0222	2017-08-05 11:05:00.000 +0300	2017-08-05 11:06:00.000 +0300	2017-08-05 14:30:00.000 +0300	Arrived
19	553	PG0222	2017-08-12 11:05:00.000 +0300	2017-08-12 11:08:00.000 +0300	2017-08-12 14:36:00.000 +0300	Arrived
20	573	PG0222	2017-08-06 11:05:00.000 +0300	2017-08-06 11:08:00.000 +0300	2017-08-06 14:35:00.000 +0300	Arrived
21	608	PG0202	2017-08-06 10:40:00.000 +0300	2017-08-06 13:31:00.000 +0300	2017-08-06 14:26:00.000 +0300	Arrived
22	701	PG0202	2017-08-12 10:40:00.000 +0300	2017-08-12 10:43:00.000 +0300	2017-08-12 11:37:00.000 +0300	Arrived
23	713	PG0202	2017-08-13 10:40:00.000 +0300	2017-08-13 10:43:00.000 +0300	2017-08-13 11:38:00.000 +0300	Arrived
24	720	PG0202	2017-08-05 10:40:00.000 +0300	2017-08-05 10:43:00.000 +0300	2017-08-05 11:37:00.000 +0300	Arrived
25	747	PG0273	2017-08-12 09:50:00.000 +0300	2017-08-12 09:52:00.000 +0300	2017-08-12 11:51:00.000 +0300	Arrived
26	749	PG0273	2017-08-13 09:50:00.000 +0300	2017-08-13 09:52:00.000 +0300	2017-08-13 11:53:00.000 +0300	Arrived
27	800	PG0273	2017-08-05 09:50:00.000 +0300	2017-08-05 09:54:00.000 +0300	2017-08-05 11:55:00.000 +0300	Arrived
28	813	PG0273	2017-08-06 09:50:00.000 +0300	2017-08-06 09:55:00.000 +0300	2017-08-06 11:54:00.000 +0300	Arrived
29	854	PG0216	2017-08-05 14:10:00.000 +0300	2017-08-05 14:11:00.000 +0300	2017-08-05 15:16:00.000 +0300	Arrived
30	881	PG0216	2017-08-06 14:10:00.000 +0300	2017-08-06 14:14:00.000 +0300	2017-08-06 15:19:00.000 +0300	Arrived
31	893	PG0216	2017-08-13 14:10:00.000 +0300	2017-08-13 14:15:00.000 +0300	2017-08-13 15:19:00.000 +0300	Arrived
32	910	PG0216	2017-08-12 14:10:00.000 +0300	2017-08-12 14:13:00.000 +0300	2017-08-12 15:18:00.000 +0300	Arrived

Обновить | Save | Cancel | Экспорт данных ... | 200 | 400+ | 400 строк получено - 0,012s (0,003s получ.),

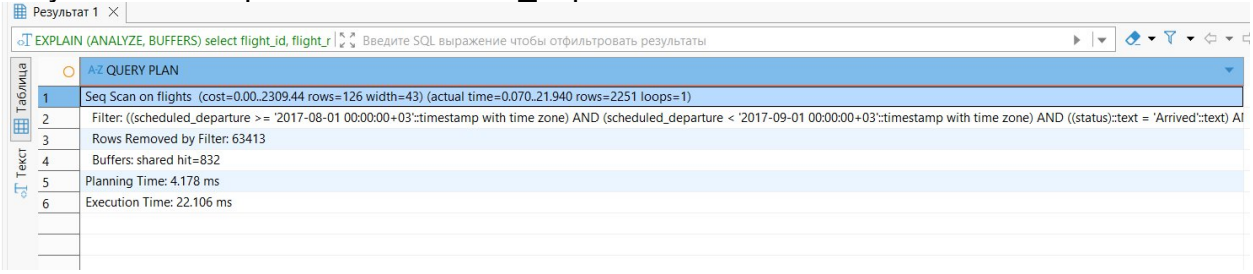
Seq Scan — вся таблица читается целиком, фильтр применяется к каждой строке. Rows Removed by Filter — сколько строк отброшено после чтения.



Оптимизированный



заранее вычисляем конкретные диапазоны дат для всех суббот и воскресений августа 2017 и сравнить scheduled_departure с ними обычными >= / <.



Оба запроса возвращают одинаковый результат -- 2251 строка , оба выполняются через Seq Scan, поскольку индекс по f.scheduled_departure отсутствует. Разница

проявляется в Execution Time: оптимизированный запрос с диапазоном дат выполняется за 22 мс, а неоптимизированный с тремя EXTRACT за 35 мс. Причина в том, что EXTRACT(year ...), EXTRACT(month ...) и EXTRACT(dow ...) вызывают функцию для каждой из 65 664 строк таблицы, тогда как диапазонное условие scheduled_departure >= '2017-08-01' AND scheduled_departure < '2017-09-01' -- это два простых сравнения timestamp плюс один EXTRACT(dow ...) только для строк, прошедших по дате, за счёт короткого замыкания AND. После создания индекса f.scheduled_departure_idx оптимизированный запрос перейдёт на Index Scan с Index Cond по диапазону и будет читать только релевантные страницы, а неоптимизированный по-прежнему останется на Seq Scan, так как функция от столбца не позволяет использовать индекс.

Краткое резюме:

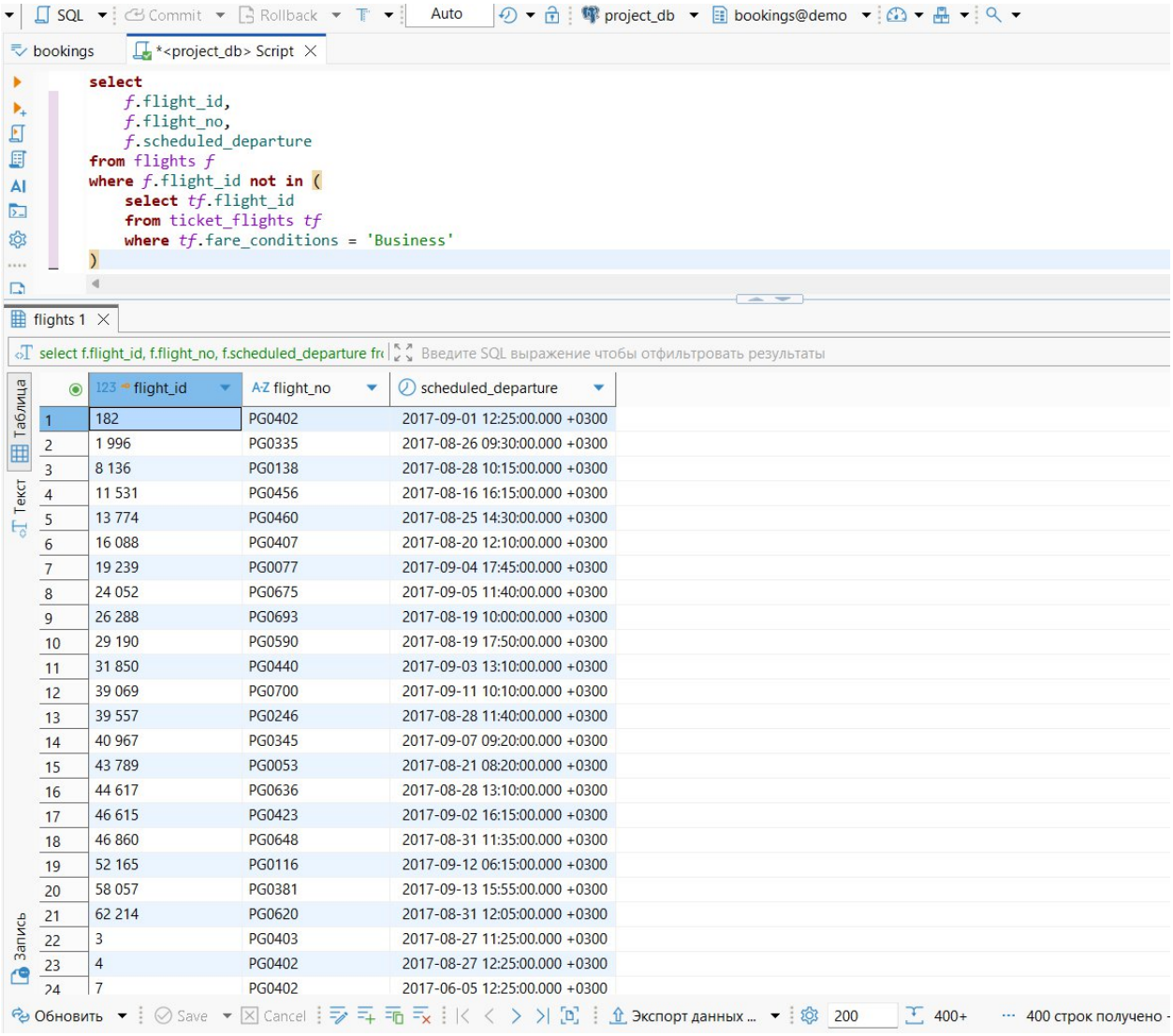
Работал ли с диксом: в обоих запросов указано Buffers: shared hit=832, при этом shared read равно нулю -- значит, все нужные страницы таблицы flights уже находились в кеше, и чтения с диска не происходило.

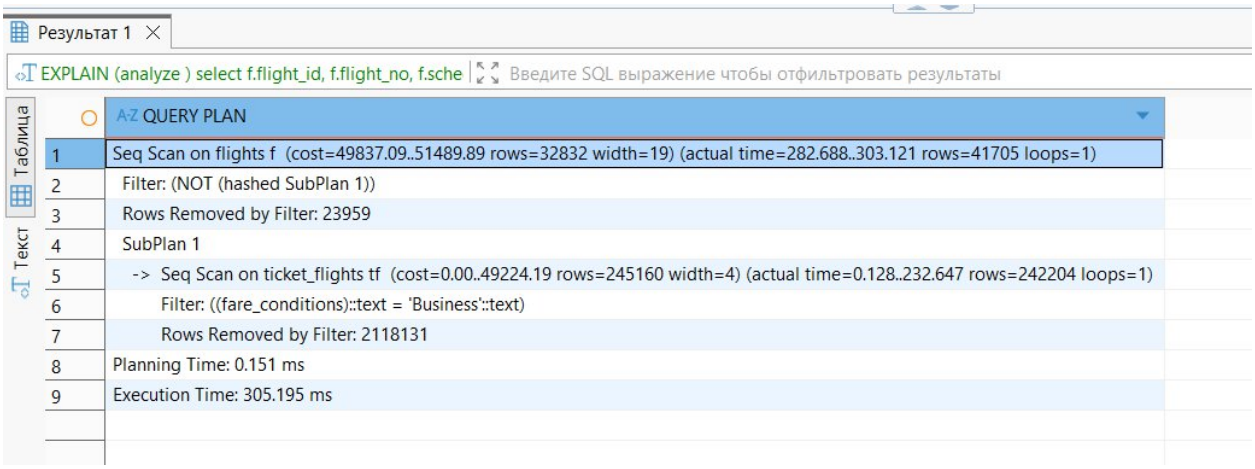
Работал ли запрос с кешем: да, оба запроса полностью работали с кешем — 832 страницы были взяты из shared buffers, что подтверждается отсутствием параметра shared read в выводе.

Использовал ли индекс: нет, ни один из запросов индекс не использовал — в обоих случаях выбран Seq Scan, то есть последовательное чтение всей таблицы flights, поскольку индекс по столбцу scheduled_departure отсутствует, а предикаты с EXTRACT(...) в принципе не позволяют его применить (не sargable).

4

not in





Not exists

```sql
--EXPLAIN (analyze, buffers )
select
    f.flight_id,
    f.flight_no,
    f.scheduled_departure
from flights f
where not exists (
    select 1
    from ticket_flights tf
    where tf.flight_id = f.flight_id
      and tf.fare_conditions = 'Business'
)
```

flights 1

select f.flight_id, f.flight_no, f.scheduled_departure fr | Введите SQL выражение чтобы отфильтровать результаты

	flight_id	flight_no	scheduled_departure
1	2 026	PG0335	2017-08-24 09:30:00.000 +0300
2	46 770	PG0648	2017-06-11 11:35:00.000 +0300
3	59 315	PG0385	2017-08-28 19:30:00.000 +0300
4	55 290	PG0354	2017-06-18 08:45:00.000 +0300
5	56 414	PG0431	2017-05-29 15:55:00.000 +0300
6	49 059	PG0111	2017-06-05 16:20:00.000 +0300
7	28 365	PG0653	2017-06-03 11:25:00.000 +0300
8	23 772	PG0621	2017-07-17 16:05:00.000 +0300
9	9 243	PG0234	2017-08-05 15:15:00.000 +0300
10	39 349	PG0555	2017-07-13 09:30:00.000 +0300
11	41 921	PG0566	2017-06-28 10:25:00.000 +0300
12	41 308	PG0001	2017-07-01 15:15:00.000 +0300
13	54 793	PG0082	2017-09-03 11:00:00.000 +0300
14	39 688	PG0038	2017-06-23 10:05:00.000 +0300
15	33 463	PG0009	2017-08-22 12:10:00.000 +0300
16	46 325	PG0479	2017-08-03 09:00:00.000 +0300
17	48 678	PG0464	2017-06-19 13:35:00.000 +0300
18	56 196	PG0514	2017-06-26 14:50:00.000 +0300
19	36 049	PG0399	2017-07-06 08:40:00.000 +0300
20	31 669	PG0056	2017-08-22 16:30:00.000 +0300
21	25 217	PG0130	2017-05-22 08:40:00.000 +0300
22	20 943	PG0295	2017-05-17 16:20:00.000 +0300
23	58 444	PG0310	2017-06-18 09:50:00.000 +0300

Обновить | Save | Cancel | Экспорт данных ... | 200 | 200+ | 200 строк получено

EXPLAIN (analyze, buffers) select f.flight_id, f.flight_n | Введите SQL выражение чтобы отфильтровать результаты

	QUERY PLAN
1	Hash Right Anti Join (cost=2309.44..53322.60 rows=38087 width=19) (actual time=316.945..324.332 rows=41705 loops=1)
2	Hash Cond: (tf.flight_id = f.flight_id)
3	Buffers: shared hit=993 read=19559
4	-> Seq Scan on ticket_flights tf (cost=0.00..49224.19 rows=245160 width=4) (actual time=0.113..234.927 rows=242204 loops=1)
5	Filter: ((fare_conditions)::text = 'Business'::text)
6	Rows Removed by Filter: 2118131
7	Buffers: shared hit=161 read=19559
8	-> Hash (cost=1488.64..1488.64 rows=65664 width=19) (actual time=23.537..23.538 rows=65664 loops=1)
9	Buckets: 131072 Batches: 1 Memory Usage: 4615kB
10	Buffers: shared hit=832
11	-> Seq Scan on flights f (cost=0.00..1488.64 rows=65664 width=19) (actual time=0.016..7.213 rows=65664 loops=1)
12	Buffers: shared hit=832
13	Planning:
14	Buffers: shared hit=7
15	Planning Time: 0.222 ms
16	Execution Time: 326.419 ms

Оптимизатор превращает NOT EXISTS в Anti Join -- соединение, которое сразу отбрасывает строки, для которых нашлось совпадение справа.

PostgreSQL строит хеш по ticket_flights (с фильтром fare_conditions = 'Business'), затем одним проходом сканирует flights и для каждого flight_id проверяет наличие в хеше. Не нужно материализовать полный список flight_id из подзапроса. Допустим

если есть индекс по ticket_flights.flight_id, план становится Nested Loop Anti Join + Index Scan -- проверка за O(log N).Хеш строится только по нужной стороне.

NOT IN → hashed SubPlan

Сначала полностью собирается множество flight_id из подзапроса, потом по нему фильтруется flights. Хеш может не влезть в work_mem и уйти на диск -- запрос тормозит. Индекс почти не используется -- всё равно нужно собрать весь список.

Если в подзапросе появится хотя бы один NULL (например, flight_id в ticket_flights nullable, или условие не отсекло NULL), то NOT IN вернёт пустой результат для всей таблицы. Значит, всё выражение никогда не станет TRUE, и ни одна строка не пройдёт фильтр.

Краткое резюме:

Работал ли запрос с диском: да, при первом запуске в буферах будет shared read (страницы читаются с диска), при повторном прогоне те же страницы уже в кеше, и shared read пропадёт.

Работал ли запрос с кешем: да, shared hit присутствует -- часть страниц (или все, если запрос запускается повторно) берётся из разделяемых буферов, без обращения к диску.

Использовал ли индекс: нет, в обоих вариантах (NOT IN и NOT EXISTS) план содержит Seq Scan по flights и ticket_flights — индекс не задействован, потому что по ticket_flights.flight_id его может не быть, а fare_conditions = 'Business' имеет низкую селективность, и оптимизатору выгоднее прочитать таблицу целиком.